



# UNIVERSITÀ DI PISA

Dipartimento di Informatica  
Corso di Laurea Triennale in Informatica

Corso a Libera Scelta - 6 CFU

## Interazione Uomo-Macchina

**Professori:**

Prof. Daniele Mazzei  
Tommaso Turchi  
Roberto Figliè

**Autore:**

Filippo Ghirardini

---

Anno Accademico 2023/2024

# Contents

|          |                                          |          |
|----------|------------------------------------------|----------|
| <b>1</b> | <b>Introduzione</b>                      | <b>3</b> |
| 1.1      | Progettazione . . . . .                  | 3        |
| 1.1.1    | Design . . . . .                         | 3        |
| 1.1.2    | Pensiero computazionale . . . . .        | 4        |
| 1.1.3    | Confronto . . . . .                      | 4        |
| <b>2</b> | <b>Persone</b>                           | <b>5</b> |
| 2.1      | Human Centered Design . . . . .          | 5        |
| 2.1.1    | Fasi di un processo . . . . .            | 5        |
| 2.1.2    | Activity Centered Design . . . . .       | 5        |
| 2.2      | Progettazione dell'interazione . . . . . | 6        |
| 2.2.1    | Understanding . . . . .                  | 6        |
| 2.2.2    | Discoverability . . . . .                | 6        |

# Interazione Uomo-Macchina

Realizzato da: Ghirardini Filippo

A.A. 2025-2026

---

# 1 Introduzione

L'obiettivo del corso è di studiare gli strumenti necessari a comprendere e gestire il processo di sviluppo delle interfacce e dei prodotti interattivi. È fondamentale che l'informatico crei un prodotto **apprezzato** dalle persone.

## 1.1 Progettazione

### 1.1.1 Design

**Definizione 1.1.1** (Design). *Per design si intende sia il processo di **progettazione** e **pianificazione** sia l'**output** stesso di questo processo.*

Nel design il primo passo è capire **perché** un problema esiste e se ha davvero bisogno di una soluzione. È quindi fondamentale fare in modo che il software risolva i problemi dell'utente, senza aggiungere funzionalità inutili non necessarie all'utente.

Nel design del prodotto è spesso necessario modificare requirement e specifiche per andare in contro alle esigenze degli utenti, rendendo quindi lo sviluppo un processo **antropocentrico** di adattamento del software.

Esistono varie sotto discipline del design, tra cui:

- **Interaction design**: l'attività di progettazione dell'interazione che avviene tra esseri umani e oggetti in generale. Ha come obiettivo quello di rendere macchine, servizi e sistemi **usabili** per gli utenti target, mettendo al centro i loro **bisogni**. Si suddivide in:

- Design di **prodotto**: progettazione di beni e servizi il cui obiettivo principale è quello di essere utilizzati da quanti più utenti possibili migliorandone la vita<sup>1</sup>

**Definizione 1.1.2** (Designer). *L'inventore o designer del prodotto si focalizza su un problema da risolvere e inventa un prodotto che grazie alle sue caratteristiche tecniche permette di risolvere un problema.*

- Design dell'**esperienza utente** o **UX**: il processo volto ad aumentare la soddisfazione e la fedeltà del cliente migliorando l'**usabilità** e la piacevolezza. Aggiunge alla tradizionale progettazione gli aspetti di business, marketing e sviluppo prodotto.

**Definizione 1.1.3** (UX Designer). *Lo UX Designer ha l'obiettivo di far vivere all'utente del suo prodotto la miglior esperienza possibile, evitando di farlo sentire frustrato e deluso.*

- Design dell'**interfaccia** o **UI**: ha come focus il modo in cui le persone interagiscono con la tecnologia e cerca di migliorare la loro comprensione di ciò che si può fare. Dalla UX si crea uno schema di interazione che serve a progettare l'interfaccia utente al fine di abilitare l'esperienza progettata.

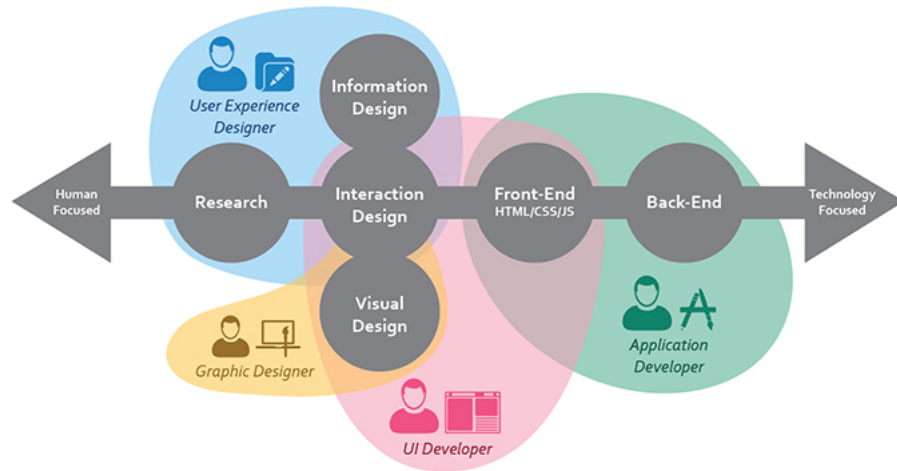
**Definizione 1.1.4** (UI Designer). *Lo UI Designer progetta l'aspetto **estetico** e la struttura dell'interfaccia così che questa durante l'utilizzo induca l'utente nel seguire l'esperienza progettata. Produce un **wireframe**<sup>2</sup> dell'interfaccia e delle linee guida che verranno seguite dagli **UI Developer** per l'implementazione.*

- **Human Machine e Human Computer Interaction**: ha come obiettivo quello di rendere possibile e **facilitare** al massimo per un essere umano l'**uso** e l'**interazione** con i sistemi del mondo IT

---

<sup>1</sup>Spesso usato in modo interscambiabile con **design industriale**: processo strategico di risoluzione dei problemi che guida l'innovazione e porta a una migliore qualità della vita attraverso prodotti, sistemi, servizi ed esperienze innovative

<sup>2</sup>Bozza grafica



### 1.1.2 Pensiero computazionale

Nel mondo del *design*, in particolare quello industriale, ci si approccia alla progettazione e risoluzione dei problemi in modo diverso dal mondo informatico.

Per questo motivo nel 2006 Jeannette Wing, direttrice del dipartimento di Informatica della Mellon University, formula la seguente definizione:

**Definizione 1.1.5** (Pensiero computazionale). *Il pensiero computazionale è un processo di formulazione di problemi e soluzioni in una forma che sia eseguibile da un agente che processi informazioni.*

Questo consiste non solo nel saper programmare ma anche nel pensare a diversi **livelli di astrazione**, abituando al *rigore* e rendendo possibili gli *atti creativi*. Permette di interagire con persone e strumenti e di usare le macchine come oggetti capaci di compensare le lentezze o l'imprecisione dell'uomo.

### 1.1.3 Confronto

Se nel **design** la progettazione è affrontata in maniera globale, nel tentativo di comprendere il problema nel suo insieme, nel **pensiero computazionale** si basa sulla suddivisione di un problema in sotto-problemi in modo da giungere ad una formulazione del problema come *algoritmo*.

È importante evidenziare che il design dell'interazione e il pensiero computazionale non sono mutualmente esclusivi ma anzi è nell'**unione** dei due e nella loro **integrazione** che si crea software di qualità.

## 2 Persone

### 2.1 Human Centered Design

I progettisti devono produrre cose che soddisfino i bisogni della gente in termini di funzioni, facilità d'uso e gratificazione emotiva, ovvero come un'esperienza **totale**.

L'obiettivo è quello di contrastare la sempre maggiore frustrazione dovuta alla complessità degli oggetti quotidiani: se le macchine hanno una modalità di funzionamento *logica*, gli umani sono l'opposto e di conseguenza nell'interazione tra i due si va a creare una relazione fra specie diverse con modalità di pensiero e funzionamento opposte.

**Definizione 2.1.1** (Human Centered Design). *È una **forma di pensiero** per lo sviluppo dei sistemi interattivi che mira a renderli **utilizzabili** e **utili** concentrandosi sugli utenti e i loro bisogni. Questo approccio migliora l'**efficacia**, l'**efficienza**, la **soddisfazione dell'utente**, l'**accessibilità** ed evitando effetti negativi sulla salute, sicurezza e performance.*

Nel design **antropocentrico** si inverte il paradigma di progettazione (*tecno-centrico*) e si mette l'utente e i suoi bisogno al centro del processo, invece delle funzionalità del prodotto. Per farlo è fondamentale **capire le persone** osservandole, dato che spesso loro stessi non sanno di cosa hanno bisogno.

È soprattutto fondamentale la **comunicazione**, specialmente dalla macchina all'utente e in particolare quando qualcosa va storto in modo che l'utente venga guidato nella risoluzione e gestione dell'errore.

L'obiettivo deve essere quello di creare **empatia** nell'utente. I ricordi hanno la capacità di far provare emozioni più profonde rispetto al presente e un utente che ha memoria di un successo favorito dalla tecnologia svilupperà empatia e bisogno per quel prodotto.

**Esempio 2.1.1** (Bitcoin). La tecnologia *blockchain* è nata in quanto c'era un bisogno per un metodo alternativo di scambiare moneta (cryptomoneta). In seguito si è provato ad applicare questa nuova tecnologia ad altri ambiti ma senza successo in quanto non si stava creando una soluzione ad un problema ma si stava cercando un problema per una soluzione.

#### 2.1.1 Fasi di un processo

Un processo HCD può essere schematizzato come un flusso continuo ed iterativo che attraversa le seguenti fasi:

- Specificare il **contesto d'uso**: identificare la user base, per cosa utilizzeranno il prodotto e sotto quali condizioni e vincoli
- Specificare i **requirements**: identificare i business requirements e gli obiettivi utente che devono essere raggiunti utilizzando il software
- **Progettare la soluzione**: può essere divisa in più sotto fasi iterative, tipicamente dalle *bozze* ai *prototipi* e alla *soluzione*
- **Testare e valutare**: fondamentale per iniziare di nuovo il ciclo in base ai risultati e procedere quindi ad un miglioramento incrementale

*Note 2.1.1.* È fondamentale implementare nel software sistemi di **tracking** dell'utente finalizzati alla produzione di statistiche di utilizzo.

#### 2.1.2 Activity Centered Design

È un'estensione di HCD e consiste nel mettere enfasi sulle attività che un utente potrebbe fare con una determinata tecnologia.

I **controlli** Activity Centered (e.g. interruttore per accendere il proiettore) sono eccellenti in teoria ma se realizzati male crea molte difficoltà. Devono quindi essere **User-Activity-Centered** e non Device-Centered

## 2.2 Progettazione dell'interazione

Il problema principale è che il *buon design* non è universale in quanto un prodotto non può essere apprezzato da tutti perché l'esperienza è soggettiva e dipende dalla persona.

Le macchine sono limitate a seguire **regole fisse e rigide** e si comportano come programmate anche se l'utente si discosta leggermente da esse, nonostante questo possa significare fare qualcosa di totalmente insensato. Questo perché, a differenza degli umani, non hanno il **buon senso** e inoltre spesso le regole da esse seguite sono conosciute solo dalla macchina e dai designers.

Sviluppatori ed ingegneri tendono a compiere l'errore di pensare che la spiegazione logica sia sufficiente per avere un buon design, mentre ovviamente non è così.

**Esempio 2.2.1** (Three Mile Island accident). L'incidente al reattore nucleare fu causato da un pessimo sviluppo dell'interfaccia di controllo: nonostante una valvola fosse bloccata in posizione aperta, il led che doveva indicare se fosse chiusa era acceso. Gli operatori impiegarono diverse ore prima di scoprire che indicava solo se il solenoide riceveva corrente e non lo stato di chiusura.

### 2.2.1 Understanding

**Definizione 2.2.1** (Understanding). *È la capacità del prodotto di farsi usare correttamente dall'utente.*

In pratica è la proprietà associata a quanto bene un prodotto dice come si usano le sue funzioni disponibili. È necessario che risponda alle seguenti domande:

- **Come** si usa il prodotto?
- Che **funzione** ha ciascun controllo?
- Come si **combinano** i controlli?

### 2.2.2 Discoverability

**Definizione 2.2.2** (Discoverability). *È la capacità di un sistema di veicolare e comunicare i propri possibili usi all'utente.*

L'idea è quindi di far capire all'utente a cosa serve il prodotto, tipicamente lavorando sulla **visibilità**. Questo non implica che poi lo si sappia usare.

Questa caratteristica si basa su sei principi fondamentali:

- **Affordances**: il termine si riferisce alla **relazione** tra le *proprietà di un oggetto* fisico e le *capacità di un agente* che determina come un oggetto possa essere utilizzato. Il contrario è **anti-affordance**, ovvero la prevenzione dell'interazione.

**Esempio 2.2.2.** La maggior parte delle sedie possono essere trasportate da una persona (they *afford* lifting). Se una persona debole non riesce a sollevarla, it *doesn't afford* lifting.

Perché questo principio sia efficace è necessario che le affordances e le anti-affordances siano **discoverable** e **perceivable**. Un esempio negativo è il vetro.

- **Signifiers**: specifica **dove** l'azione deve essere eseguita (non quale) e comunicano quindi come usare il design. Possono essere **deliberati** ed **intenzionali**, come il segno "spingi" sulla porta, o **accidentali** e **non intenzionali**, come un percorso nell'erba causato da tante persone che ci hanno camminato.

Per passare da un'affordance a un signifier di solito si procede tramite **convenzioni**, come ad esempio il fatto che una maniglia su una porta indica che devi usarla per aprirla. L'interpretazione di una *perceived affordance* è una **convenzione culturale**.

- **Mapping:** indica la **relazione** tra gli elementi di due insiemi di cose. È importante soprattutto nel design e layout dei controlli e degli schermi in quanto utilizza **analogie spaziali** tra i controlli e i dispositivi controllati per indicare come utilizzarli (e.g. fornelli). Alcuni mappings sono biologici (muovere la mano in alto e in basso indicano più e meno) mentre altri culturali (indicatore del carburante pieno a destra o sinistra).
- **Feedback:** la comunicazione del risultato di un'azione. Deve essere **immediato** ed **informativo** senza ostruire altro (e.g. troppe notifiche ti portano a disattivarle completamente).
- **Conceptual model:** una spiegazione, di solito molto semplificata, di come un qualcosa funziona (e.g. cartelle di file sul pc). Non deve essere completa o accurata, basta che sia utile. È fondamentale che l'**assunzione** su cui si basa la semplificazione rimanga vera (e.g. far vedere i file anche se sono nel cloud, la connessione deve esserci). Sono spesso inferiti dal dispositivo stesso, tramandati da persone, estratti da manuali o costruiti con l'esperienza.  
I **mental model** sono invece come le persone rappresentano il funzionamento delle cose nella loro mente. È diverso per ognuno e una persona singola può averne anche diversi per lo stesso oggetto, a volte anche in conflitto.  
La combinazione di informazioni necessarie per la creazione di un *conceptual model* di un dispositivo con cui interagiamo è la **system image**. Ad esempio aspetto fisico, somiglianza con altri dispositivi già usati, pubblicità, articoli di giornale o manuali.  
Quando la system image è incoerente o inappropriata, l'utente ha difficoltà nell'usare il dispositivo. È importante che il designer trasmetta il suo conceptual model correttamente, tramite la system image, al conceptual model dell'utente.
- **Constraints:** servono a limitare l'insieme delle possibili azioni grazie alla conoscenza del mondo e i modelli concettuali. Possono essere
  - **Fisici**, tra cui le **forcing functions** ovvero casi estremi di constraints che evitano comportamenti inappropriati. Si suddividono in:
    - \* **Interlocks:** impone che una serie di operazioni siano eseguite nel giusto ordine
    - \* **Lock-ins:** mantiene un'operazione attiva evitando che qualcuno la interrompa prematuramente (e.g. conferma per la chiusura di un programma)
    - \* **Lockout:** evita che qualcuno acceda ad uno spazio pericoloso o evita che accada un determinato evento (e.g. limite di età)
  - **Culturali**
  - **Semantici**
  - **Logici**

*Note 2.2.1 (Consistenza).* Dato che violare delle convenzioni richiede nuovo apprendimento, mantenere una **consistenza** nei vari design è una caratteristica importante.